

Python for AI & Machine Learning

Class 19 – Modules and Packages in Python

Course: Python for AI & Machine Learning

Module: Advanced Python Programming

Duration: 2 Hours (Theory + Practical)

Learning Objectives

By the end of this session, students will be able to:

- Understand what Modules and Packages are.
 - Differentiate between Built-in, User-defined, and Third-party Modules.
 - Create and use custom modules.
 - Import modules using different techniques.
 - Explore commonly used Python modules.
 - Understand Packages and their structure.
 - Install external libraries using `pip`.
 - Learn the importance of modules and packages in AI & Machine Learning.
-

Revision

In the previous class, we learned about **File Handling**, including:

- Opening and closing files
- Reading and writing text files
- Appending data
- Working with CSV files
- Working with JSON files
- Exception handling with files

Today, we will learn how Python organizes reusable code using **Modules and Packages**.

Introduction to Modules

As applications grow larger, writing all the code in a single file becomes difficult to manage.

Python solves this problem using **Modules**.

A **Module** is simply a Python file (`.py`) that contains reusable code such as:

- Functions
- Variables
- Classes
- Constants

Modules help organize code into smaller, manageable files.

Why Do We Need Modules?

Without Modules:

- Code becomes lengthy.
- Difficult to debug.
- Difficult to maintain.
- Code duplication increases.

With Modules:

- Better organization
 - Code reusability
 - Easy maintenance
 - Easier debugging
 - Faster development
 - Team collaboration becomes easier
-

Types of Modules

Python provides three types of modules:

1. Built-in Modules

These are provided by Python itself.

Examples:

- `math`
- `random`
- `datetime`
- `os`
- `sys`
- `time`
- `json`
- `csv`

No installation is required.

2. User-defined Modules

Modules created by programmers for their own applications.

Example:

```
calculator.py
```

This file itself becomes a module.

3. Third-party Modules

These are developed by the Python community and installed separately.

Examples:

- NumPy
- Pandas
- Matplotlib
- Scikit-learn
- TensorFlow
- OpenCV
- Flask
- Django

Installation is done using:

```
pip install package_name
```

Importing Modules

Python provides several ways to import modules.

Method 1 – Import Entire Module

```
import math

print(math.sqrt(25))
print(math.factorial(5))
```

Output

```
5.0  
120
```

Method 2 – Import Specific Functions

```
from math import sqrt  
  
print(sqrt(64))
```

Output

```
8.0
```

Method 3 – Import Multiple Functions

```
from math import sqrt, factorial  
  
print(sqrt(81))  
print(factorial(6))
```

Method 4 – Import Everything

```
from math import *  
  
print(sqrt(49))
```

Note: This method is generally not recommended because it may create naming conflicts.

Method 5 – Import with Alias

```
import math as m  
  
print(m.pi)  
print(m.sqrt(100))
```

Aliases make code shorter and easier to read.

The Math Module

The `math` module provides mathematical functions.

Example:

```
import math

print(math.pi)
print(math.e)
print(math.sqrt(144))
print(math.pow(5, 2))
print(math.factorial(6))
print(math.ceil(5.2))
print(math.floor(5.9))
```

The Random Module

The `random` module generates random numbers.

Example:

```
import random

print(random.randint(1, 100))
```

Random Choice:

```
import random

colors = ["Red", "Blue", "Green"]

print(random.choice(colors))
```

Random Float:

```
import random

print(random.random())
```

Applications:

- AI simulations
 - Gaming
 - Password generators
 - Machine Learning dataset shuffling
-

The Datetime Module

The `datetime` module works with dates and time.

Example:

```
import datetime

today = datetime.datetime.now()

print(today)
```

Extract Current Date:

```
import datetime

print(datetime.date.today())
```

Applications:

- Logging
 - Attendance systems
 - AI prediction timestamps
 - Report generation
-

The OS Module

The `os` module interacts with the operating system.

Example:

```
import os

print(os.getcwd())
```

List Files:

```
import os

print(os.listdir())
```

Create Folder:

```
import os

os.mkdir("StudentFiles")
```

The Sys Module

The `sys` module provides access to Python runtime information.

Example:

```
import sys

print(sys.version)
```

Exit Program:

```
import sys

sys.exit()
```

Creating Your Own Module

Create a file:

```
calculator.py
```

Contents:

```
def add(a, b):
    return a + b
```

```
def multiply(a, b):  
    return a * b
```

Use the module:

```
import calculator  
  
print(calculator.add(10, 20))  
print(calculator.multiply(5, 6))
```

Output

```
30  
30
```

What is a Package?

A **Package** is a collection of related modules stored inside a folder.

Example:

```
MyProject/  
  
    calculations/  
        __init__.py  
        addition.py  
        subtraction.py  
        multiplication.py
```

Packages help organize large applications into logical sections.

The `__init__.py` File

The `__init__.py` file tells Python that a folder should be treated as a package.

Although optional in modern Python versions, it is still widely used for package initialization and better compatibility.

Importing from a Package

Folder Structure:

```
mypackage/  
    __init__.py  
    calculator.py
```

Import:

```
from mypackage import calculator
```

Installing Third-party Libraries

Python uses **pip** (Package Installer for Python).

Check pip version:

```
pip --version
```

Install a library:

```
pip install numpy
```

Upgrade a package:

```
pip install --upgrade numpy
```

Uninstall:

```
pip uninstall numpy
```

View installed packages:

```
pip list
```

Popular Python Libraries for AI & Machine Learning

Library	Purpose
NumPy	Numerical computing
Pandas	Data analysis
Matplotlib	Data visualization
Seaborn	Statistical visualization
Scikit-learn	Machine Learning algorithms
TensorFlow	Deep Learning
PyTorch	AI and Deep Learning
OpenCV	Computer Vision
NLTK	Natural Language Processing
Transformers	Large Language Models (LLMs)

AI Example 1 – NumPy

```
import numpy as np

numbers = np.array([10, 20, 30])

print(numbers)
```

AI Example 2 – Pandas

```
import pandas as pd

students = {
    "Name": ["Rahul", "Sneha"],
    "Marks": [95, 90]
}

df = pd.DataFrame(students)

print(df)
```

AI Example 3 – Random Dataset Selection

```
import random

dataset = ["Image1", "Image2", "Image3"]

print(random.choice(dataset))
```

AI Example 4 – Date Logging

```
import datetime

print(datetime.datetime.now())
```

Useful for:

- Model training logs
- Experiment tracking
- Prediction history

Best Practices

- Import only what you need.
- Use aliases for long module names (`numpy as np` , `pandas as pd`).
- Organize large projects using packages.
- Avoid using `from module import *`.
- Keep user-defined modules focused on a single purpose.
- Use virtual environments for large projects.

Practical Exercise 1

Use the `math` module to calculate:

- Square root
 - Factorial
 - Power
 - Ceiling value
 - Floor value
-

Practical Exercise 2

Generate:

- A random number
 - A random color
 - A random password using the `random` module
-

Practical Exercise 3

Create your own module called `calculator.py`.

Implement:

- Addition
- Subtraction
- Multiplication
- Division

Import the module into another Python file and use all functions.

Practical Exercise 4

Create a package called `student`.

Inside it, create:

- `attendance.py`
- `marks.py`
- `fees.py`

Import each module into a main program.

Practical Exercise 5

Install the following libraries using `pip`:

- NumPy
- Pandas
- Matplotlib

Verify the installation by importing each library.

Assignment

AI Student Analytics System

Develop a project that:

- Creates a custom module for student calculations.
 - Uses the `random` module to assign practice questions.
 - Uses the `datetime` module to log submission time.
 - Stores data using the `json` module.
 - Uses NumPy to perform simple numerical calculations.
 - Uses Pandas to display student records in tabular format.
-

Interview Questions

1. What is a Module in Python?
 2. What are the advantages of using Modules?
 3. What are the different types of Modules?
 4. How do you create a user-defined Module?
 5. What is a Package?
 6. What is the purpose of the `__init__.py` file?
 7. What is `pip`?
 8. What is the difference between a Module and a Package?
 9. Why are aliases used while importing modules?
 10. Name five popular Python libraries used in AI and Machine Learning.
 11. Explain the purpose of the `math`, `random`, and `datetime` modules.
 12. How are Modules and Packages used in real-world AI projects?
-

Summary

In today's session, we learned:

- Introduction to Modules
- Types of Modules
- Importing Modules
- Built-in Modules
- User-defined Modules
- Third-party Modules
- The `math`, `random`, `datetime`, `os`, and `sys` modules
- Creating Custom Modules
- Packages and `__init__.py`
- Installing Libraries using `pip`
- AI & Machine Learning Libraries
- Practical Exercises
- Assignment

Modules and Packages are essential for writing clean, reusable, and scalable Python applications. They form the foundation of modern software development and are extensively used in AI, Machine Learning, Data Science, Web Development, and Automation.

Next Class Preview

Class 20 – Introduction to NumPy for AI & Machine Learning

Topics to be covered:

- Introduction to NumPy
- Installing NumPy
- Creating Arrays
- Array Operations
- Indexing and Slicing
- Mathematical Operations
- Statistical Functions
- Reshaping Arrays
- NumPy vs Python Lists
- AI & Machine Learning Applications of NumPy